



MANUAL TÉCNICO

Sistema de Gestión Organizacional

Aplicación de escritorio desarrollada en Java SE con enfoque en Programación Orientada a Objetos

Java SE 17 • Swing • SQLite • JDBC • Maven • iText 5

GRUPO #3

Proyecto Final — Programación Orientada a Objetos (POO)

Versión del documento: 1.0

Tabla de contenido

1. Introducción	4
1.1 Alcance	4
1.2 Público objetivo	4
2. Objetivos del sistema	5
2.1 Objetivo general	5
2.2 Objetivos específicos	5
3. Tecnologías y herramientas utilizadas	6
3.1 Requisitos del entorno de desarrollo y ejecución	6
4. Arquitectura del sistema	7
4.1 Flujo de una operación típica	7
4.2 Diagrama de capas	7
5. Estructura del proyecto	8
6. Modelo de base de datos	9
6.1 Entidades y relaciones	9
6.2 Diccionario de datos	9
Tabla: pais	9
Tabla: departamento	9
Tabla: cargo	9
Tabla: empleado	9
Tabla: proyecto	10
Tabla: asignacion	10
Tabla: usuarios	10
7. Principios de Programación Orientada a Objetos aplicados	11
7.1 Encapsulamiento	11
7.2 Herencia	11
7.3 Polimorfismo	11
7.4 Composición y agregación	11
7.5 Patrón Singleton	11
7.6 Arquitectura en capas (MVC)	11
7.7 Manejo de excepciones personalizado	11
8. Descripción detallada de paquetes y clases	13
8.1 Paquete org.proyecto	13
Main.java	13
8.2 Paquete org.proyecto.conexion	13
ConexionSQLite.java	13
8.3 Paquete org.proyecto.modelo	13
8.4 Paquete org.proyecto.dao	14
8.5 Paquete org.proyecto.controlador	14

ValidacionException.java	14
8.6 Paquete org.proyecto.vista	15
9. Interfaz gráfica y navegación	16
9.1 Flujo de navegación	16
9.2 Módulo de exportación	16
10. Manual de instalación y despliegue.....	17
10.1 Requisitos previos	17
10.2 Compilación del proyecto	17
10.3 Ejecución de la aplicación	17
10.4 Primer inicio	17
10.5 Ubicación de la base de datos	17
11. Manejo de errores y validaciones.....	18
11.1 Ejemplo de manejo en la capa de vista.....	18
12. Conclusiones.....	19

1. Introducción

Este manual técnico documenta el diseño, la arquitectura y la implementación del Sistema de Gestión Organizacional, una aplicación de escritorio desarrollada en Java SE que permite administrar países, departamentos, cargos, empleados, proyectos y la asignación de empleados a proyectos. El sistema fue construido como proyecto final de la asignatura de Programación Orientada a Objetos (POO) y aplica de forma práctica los principios fundamentales del paradigma: encapsulamiento, herencia, polimorfismo y composición, junto con patrones de diseño como Singleton y una arquitectura en capas inspirada en el patrón Modelo-Vista-Controlador (MVC).

El documento está dirigido a desarrolladores, evaluadores académicos y cualquier persona que necesite comprender, mantener, ampliar o desplegar el sistema. Incluye la descripción de la arquitectura, el modelo de datos, el detalle de cada paquete y clase, los principios de POO aplicados, y la guía de instalación y ejecución.

1.1 Alcance

El manual cubre la totalidad del código fuente entregado: la capa de conexión a base de datos, los modelos de dominio, los objetos de acceso a datos (DAO), los controladores de negocio y la capa de presentación gráfica (Swing), así como los módulos de exportación de información a formatos CSV y PDF.

1.2 Público objetivo

- Estudiantes y docentes que evalúen el cumplimiento de los principios de POO.
- Desarrolladores que requieran dar mantenimiento o extender el sistema.
- Cualquier usuario técnico que necesite instalar y ejecutar la aplicación.

2. Objetivos del sistema

2.1 Objetivo general

Proveer una herramienta de escritorio que permita a una organización gestionar de forma centralizada su información de países, departamentos, cargos, empleados y proyectos, así como las asignaciones de personal a proyectos, garantizando la integridad de los datos mediante validaciones de negocio y persistencia en una base de datos relacional.

2.2 Objetivos específicos

- Permitir el registro, actualización, consulta y eliminación (CRUD) de países, departamentos, cargos, empleados y proyectos.
- Gestionar la relación muchos-a-muchos entre empleados y proyectos mediante el módulo de Asignaciones, controlando las horas asignadas.
- Restringir el acceso al sistema mediante un módulo de autenticación de usuarios (login).
- Permitir la exportación de la información visualizada en tablas a formatos CSV y PDF.
- Aplicar de manera demostrable los pilares de la Programación Orientada a Objetos: herencia, polimorfismo, encapsulamiento y composición.
- Mantener una arquitectura en capas que separe la persistencia, la lógica de negocio y la interfaz gráfica, facilitando el mantenimiento del sistema.

3. Tecnologías y herramientas utilizadas

El sistema fue construido íntegramente con tecnologías del ecosistema Java, sin dependencias de frameworks externos de interfaz gráfica, lo cual refuerza el enfoque académico en POO puro.

Tecnología / Herramienta	Versión	Rol dentro del proyecto
Java SE (JDK)	17	Lenguaje y plataforma de ejecución del proyecto.
Swing (javax.swing)	Incluida en el JDK	Construcción de la interfaz gráfica de escritorio (MDI).
SQLite	Motor embebido	Base de datos relacional local, almacenada en el archivo organizacion.db.
sqlite-jdbc (Xerial)	3.45.1.0	Driver JDBC para conectar la aplicación con SQLite.
iTextPDF	5.5.13.3	Generación de reportes en formato PDF a partir de tablas.
Apache Maven	IDE 30	Gestión de dependencias, compilación y empaquetado del proyecto (JAR ejecutable).
maven-shade-plugin	3.5.1	Empaqueta la aplicación junto con sus dependencias en un único JAR ejecutable.

3.1 Requisitos del entorno de desarrollo y ejecución

- JDK 17 o superior instalado y configurado en el PATH del sistema.
- Apache Maven 3.8 o superior.
- Sistema operativo con soporte para interfaz gráfica de escritorio (Windows, Linux con entorno gráfico o macOS).
- No se requiere instalar un servidor de base de datos: SQLite trabaja de forma embebida en un único archivo.

4. Arquitectura del sistema

El sistema está organizado en capas, siguiendo una variante del patrón Modelo-Vista-Controlador (MVC), lo que permite separar responsabilidades y facilita el mantenimiento y la extensión del código. El paquete raíz de la aplicación es `org.proyecto` y se subdivide en cinco paquetes principales.

Paquete	Responsabilidad
<code>org.proyecto</code>	Contiene la clase Main, punto de entrada de la aplicación.
<code>org.proyecto.conexion</code>	Gestiona la conexión JDBC a SQLite mediante el patrón Singleton (<code>ConexionSQLite</code>).
<code>org.proyecto.modelo</code>	Clases de dominio (entidades) que representan los objetos del negocio: Persona, Empleado, Pais, Departamento, Cargo, Proyecto, Asignacion y Empresa.
<code>org.proyecto.dao</code>	Objetos de Acceso a Datos (Data Access Object): encapsulan las sentencias SQL (INSERT, UPDATE, DELETE, SELECT) para cada entidad.
<code>org.proyecto.controlador</code>	Contiene la lógica de negocio y las validaciones; actúa como intermediario entre la vista y el DAO.
<code>org.proyecto.vista</code>	Componentes de interfaz gráfica Swing: ventana de login, ventana principal MDI, ventanas internas por módulo y utilidades de exportación.

4.1 Flujo de una operación típica

Cuando el usuario interactúa con un formulario (por ejemplo, para registrar un nuevo Empleado), el flujo de datos atraviesa las capas de la siguiente manera:

```

Vista (EmpleadoInterna)
  | captura los datos del formulario y construye un objeto Empleado
  ▼
Controlador (EmpleadoControlador)
  | valida las reglas de negocio (campos obligatorios, rangos, duplicados)
  ▼
DAO (EmpleadoDAO)
  | traduce la operación a sentencias SQL con PreparedStatement
  ▼
Conexión (ConexionSQLite)
  | entrega la Connection JDBC activa (Singleton)
  ▼
Base de datos SQLite (organizacion.db)
  
```

4.2 Diagrama de capas

La siguiente tabla resume la dirección de las dependencias entre capas: cada capa solo conoce a la inmediatamente inferior, lo cual reduce el acoplamiento.

Capa	Depende de	No depende de
Vista	Controlador, Modelo	DAO, Conexión
Controlador	DAO, Modelo	Vista
DAO	Conexión, Modelo	Vista, Controlador
Modelo	(ninguna capa del sistema)	Vista, Controlador, DAO, Conexión

5. Estructura del proyecto

El proyecto sigue la estructura estándar de Maven. A continuación se muestra el árbol de directorios relevante:

```

ProyectoFinalP00/
├── pom.xml                      Configuración de Maven (dependencias y build)
├── db/
│   └── schema.sql              Script SQL de referencia del esquema
├── organizacion.db              Base de datos SQLite (se genera/usa en tiempo de
ejecución)
├── src/main/java/org/proyecto/
│   ├── Main.java               Punto de entrada de la aplicación
│   ├── conexion/
│   │   └── ConexionSQLite.java Singleton de conexión JDBC
│   ├── modelo/
│   │   ├── Persona.java        Clase abstracta base
│   │   ├── Empleado.java       Hereda de Persona
│   │   ├── Pais.java
│   │   ├── Departamento.java
│   │   ├── Cargo.java
│   │   ├── Proyecto.java
│   │   ├── Asignacion.java      Relación N:M Empleado-Proyecto
│   │   └── Empresa.java        Entidad agregadora (composición)
│   ├── dao/
│   │   ├── PaisDAO.java, DepartamentoDAO.java, CargoDAO.java
│   │   ├── EmpleadoDAO.java, ProyectoDAO.java, AsignacionDAO.java
│   │   └── UsuarioDao.java      Autenticación
│   ├── controlador/
│   │   ├── PaisControlador.java, DepartamentoControlador.java, CargoControlador.java
│   │   ├── EmpleadoControlador.java, ProyectoControlador.java, AsignacionControlador.java
│   │   └── ValidacionException.java
│   └── vista/
│       ├── FrmLogin.java        Ventana de inicio de sesión
│       ├── VentanaPrincipal.java Ventana principal MDI (JDesktopPane)
│       ├── DashboardInterna.java Panel de inicio / resumen
│       ├── PaisInterna.java, DepartamentoInterna.java, CargoInterna.java
│       ├── EmpleadoInterna.java, ProyectoInterna.java, AsignacionInterna.java
│       ├── EstiloUI.java        Constantes visuales y utilidades de estilo
│       ├── UtilidadesUI.java
│       ├── ExportadorCSV.java    Exportación de JTable a CSV
│       └── ExportadorPDF.java     Exportación de JTable a PDF (iText)

```


6. Modelo de base de datos

La persistencia se realiza sobre una base de datos SQLite embebida, almacenada en el archivo organizacion.db en la raíz del proyecto. El esquema se define en db/schema.sql y adicionalmente la clase ConexionSQLite lo crea de forma automática y programática al iniciar la aplicación (patrón "esquema autogestionado"), de modo que el sistema es funcional con solo ejecutarlo, sin pasos manuales de configuración de base de datos.

6.1 Entidades y relaciones

- país (1) —< (N) empleado
- departamento (1) —< (N) empleado
- cargo (1) —< (N) empleado
- empleado (1) —< (N) asignacion >— (N) proyecto → relación muchos a muchos resuelta con la tabla intermedia asignacion
- usuarios: tabla independiente utilizada exclusivamente para la autenticación (login).

6.2 Diccionario de datos

Tabla: país

Columna	Tipo	Restricciones
id	INTEGER	PRIMARY KEY AUTOINCREMENT
nombre	TEXT	NOT NULL, UNIQUE

Tabla: departamento

Columna	Tipo	Restricciones
id	INTEGER	PRIMARY KEY AUTOINCREMENT
nombre	TEXT	NOT NULL, UNIQUE
presupuesto	REAL	NOT NULL, DEFAULT 0

Tabla: cargo

Columna	Tipo	Restricciones
id	INTEGER	PRIMARY KEY AUTOINCREMENT
nombre	TEXT	NOT NULL, UNIQUE
salarioBase	REAL	NOT NULL, DEFAULT 0

Tabla: empleado

Columna	Tipo	Restricciones
id	INTEGER	PRIMARY KEY AUTOINCREMENT
nombre	TEXT	NOT NULL

Columna	Tipo	Restricciones
apellido	TEXT	NOT NULL
salario	REAL	NOT NULL, DEFAULT 0
idPais	INTEGER	NOT NULL, FK → pais(id) ON DELETE RESTRICT
idDepartamento	INTEGER	NOT NULL, FK → departamento(id) ON DELETE RESTRICT
idCargo	INTEGER	NOT NULL, FK → cargo(id) ON DELETE RESTRICT

Tabla: proyecto

Columna	Tipo	Restricciones
id	INTEGER	PRIMARY KEY AUTOINCREMENT
nombre	TEXT	NOT NULL, UNIQUE
presupuesto	REAL	NOT NULL, DEFAULT 0

Tabla: asignacion

Columna	Tipo	Restricciones
id	INTEGER	PRIMARY KEY AUTOINCREMENT
idEmpleado	INTEGER	NOT NULL, FK → empleado(id) ON DELETE CASCADE
idProyecto	INTEGER	NOT NULL, FK → proyecto(id) ON DELETE CASCADE
horasAsignadas	REAL	NOT NULL, DEFAULT 0

Tabla: usuarios

Columna	Tipo	Restricciones
id	INTEGER	PRIMARY KEY AUTOINCREMENT
usuario	TEXT	NOT NULL, UNIQUE
password	TEXT	NOT NULL
rol	TEXT	NOT NULL

Nota: al iniciar la aplicación por primera vez, ConexionSQLite crea automáticamente un usuario administrador por defecto (usuario: admin, contraseña: 1234, rol: Administrador) mediante la sentencia INSERT OR IGNORE, con el fin de que el sistema sea utilizable de inmediato en el primer arranque.

7. Principios de Programación Orientada a Objetos aplicados

El proyecto fue diseñado explícitamente para evidenciar los cuatro pilares de la POO, además de un patrón de diseño estructural (Singleton) y una arquitectura en capas de tipo MVC.

7.1 Encapsulamiento

Todas las clases del paquete modelo declaran sus atributos como `private` y exponen el acceso controlado mediante métodos `getter` y `setter` públicos. Esto se aplica de forma consistente en `Pais`, `Departamento`, `Cargo`, `Proyecto`, `Empleado` y `Asignacion`, protegiendo el estado interno de los objetos de modificaciones no controladas.

7.2 Herencia

La clase abstracta `Persona` define los atributos comunes `id`, `nombre` y `apellido`, así como el método abstracto `getDescripcion()`. La clase `Empleado` extiende `Persona` (`Empleado extends Persona`), heredando sus atributos y comportamientos, y añadiendo los atributos propios del dominio laboral: `salario`, `pais`, `departamento` y `cargo`.

7.3 Polimorfismo

El método abstracto `getDescripcion()`, declarado en `Persona`, es sobrescrito (`@Override`) en `Empleado` para retornar una descripción específica que combina `nombre`, `apellido` y `cargo`. Esto permite que, en tiempo de ejecución, distintas subclases de `Persona` puedan responder de forma distinta ante el mismo mensaje, un caso clásico de polimorfismo de sobrescritura (`overriding`).

7.4 Composición y agregación

La clase `Empresa` actúa como entidad agregadora: contiene listas de `Departamento`, `Empleado` y `Proyecto`, y expone métodos para agregar elementos a cada colección (`agregarDepartamento`, `agregarEmpleado`, `agregarProyecto`), representando la relación "todo-parte" propia de la composición.

De forma similar, `Empleado` está compuesto por asociaciones hacia `Pais`, `Departamento` y `Cargo`, y `Asignacion` está compuesta por referencias a `Empleado` y `Proyecto`, resolviendo la relación muchos a muchos entre ambas entidades.

7.5 Patrón Singleton

La clase `ConexionSQLite` implementa el patrón Singleton mediante un constructor privado, un atributo estático `instancia` y el método estático sincronizado `getInstancia()`, garantizando que exista una única instancia de conexión a la base de datos durante todo el ciclo de vida de la aplicación.

7.6 Arquitectura en capas (MVC)

La separación entre modelo, dao, controlador y vista refleja el patrón Modelo-Vista-Controlador: el modelo representa los datos, la vista gestiona la interacción con el usuario, y el controlador contiene la lógica de negocio y las validaciones, evitando que la vista acceda directamente a la base de datos.

7.7 Manejo de excepciones personalizado

La clase `ValidacionException`, que extiende `Exception`, se utiliza como excepción de negocio propia del sistema. Los controladores la lanzan cuando los datos ingresados no cumplen las reglas de validación (campos obligatorios, valores fuera de rango, registros duplicados), permitiendo que la capa de vista capture este tipo específico de error y lo distinga de errores técnicos (`SQLException`).

8. Descripción detallada de paquetes y clases

8.1 Paquete org.proyecto

Main.java

Clase con el método main, punto de entrada de la aplicación. Sus responsabilidades son:

- Aplicar el Look and Feel de la interfaz mediante EstiloUI.aplicarLookAndFeel().
- Forzar la inicialización de la conexión a la base de datos y del esquema mediante ConexionSQLite.getInstance().
- Lanzar la ventana de inicio de sesión (FrmLogin) dentro del hilo de eventos de Swing (SwingUtilities.invokeLater).
- Registrar un shutdown hook que cierra la conexión a la base de datos al finalizar la aplicación.
- Capturar cualquier excepción crítica de arranque y mostrarla mediante un JOptionPane, finalizando la ejecución de forma controlada (System.exit(1)).

8.2 Paquete org.proyecto.conexion

ConexionSQLite.java

Administra la conexión JDBC hacia el archivo organizacion.db. Aplica el patrón Singleton (ver sección 7.5). Además de gestionar la conexión, ejecuta inicializarEsquema(), que crea todas las tablas del sistema con sentencias CREATE TABLE IF NOT EXISTS si aún no existen, habilita las llaves foráneas (PRAGMA foreign_keys = ON) e inserta el usuario administrador por defecto.

Método	Descripción
getInstance()	Retorna la única instancia de la clase (crea la conexión si no existe).
getConnection()	Retorna el objeto Connection activo, reabriendo la conexión si estuviera cerrada.
cerrar()	Cierra la conexión a la base de datos de forma segura.

8.3 Paquete org.proyecto.modelo

Contiene las clases de dominio del sistema, todas ellas POJOs (Plain Old Java Objects) con atributos privados, constructores sobrecargados, getters/setters y sobrescritura de toString().

Clase	Tipo	Descripción
Persona	Clase abstracta	Atributos comunes id, nombre, apellido. Declara el método abstracto getDescripcion().
Empleado	Hereda de Persona	Añade salario y las asociaciones a Pais, Departamento y Cargo. Implementa getDescripcion().
Pais	Entidad	id y nombre. Relación 1:N con Empleado.
Departamento	Entidad	id, nombre y presupuesto. Relación 1:N con Empleado.
Cargo	Entidad	id, nombre y salarioBase. Relación 1:N con Empleado.
Proyecto	Entidad	id, nombre y presupuesto. Relación N:M con Empleado vía Asignacion.

Clase	Tipo	Descripción
Asignacion	Entidad de relación	Resuelve la relación N:M entre Empleado y Proyecto; incluye horasAsignadas.
Empresa	Entidad agregadora	Contiene listas de Departamento, Empleado y Proyecto (composición).

8.4 Paquete org.proyecto.dao

Los DAO encapsulan el acceso a datos mediante JDBC utilizando PreparedStatement para prevenir inyección SQL. Cada DAO expone, de forma consistente, operaciones de inserción, actualización, eliminación y listado.

Clase DAO	Entidad gestionada	Operaciones principales
PaisDAO	Pais	insertar, actualizar, eliminar, listar
DepartamentoDAO	Departamento	insertar, actualizar, eliminar, listar
CargoDAO	Cargo	insertar, actualizar, eliminar, listar
EmpleadoDAO	Empleado	insertar, actualizar, eliminar, listar (con JOIN a pais, departamento y cargo)
ProyectoDAO	Proyecto	insertar, actualizar, eliminar, listar
AsignacionDAO	Asignacion	insertar, actualizar, eliminar, listar, existeAsignacion (validación de duplicados)
UsuarioDao	Usuario (login)	validarAcceso(usuario, password)

EmpleadoDAO es representativo del patrón utilizado en toda la capa: la consulta SELECT_BASE realiza JOIN con las tablas pais, departamento y cargo para reconstruir el objeto Empleado completo (incluyendo sus asociaciones) a partir de un único ResultSet, mediante el método privado mapearEmpleado(ResultSet).

8.5 Paquete org.proyecto.controlador

Los controladores actúan como intermediarios entre la vista y el DAO, y son responsables de aplicar las reglas de validación de negocio antes de persistir cualquier cambio.

Clase	Reglas de validación destacadas
PaisControlador	Nombre obligatorio.
DepartamentoControlador	Nombre obligatorio; presupuesto no negativo.
CargoControlador	Nombre obligatorio; salario base mayor que cero.
EmpleadoControlador	Nombre y apellido obligatorios; salario mayor que cero; país, departamento y cargo obligatorios.
ProyectoControlador	Nombre obligatorio; presupuesto no negativo.
AsignacionControlador	Empleado y proyecto obligatorios; horas asignadas mayores que cero y no superiores a 200; el empleado no puede tener una asignación duplicada al mismo proyecto (se valida contra la base de datos mediante AsignacionDAO.existeAsignacion).

Todos los controladores siguen el mismo patrón para el método guardar(entidad): si el identificador de la entidad es 0 se ejecuta una inserción; en caso contrario, se ejecuta una actualización. Este criterio evita duplicar formularios de alta y edición en la capa de vista.

ValidacionException.java

Excepción de negocio (extiende `Exception`, comprobada en tiempo de compilación) lanzada por los controladores cuando los datos no cumplen las reglas descritas anteriormente. La capa de vista la captura de forma independiente de `SQLException` para mostrar mensajes claros y específicos al usuario.

8.6 Paquete `org.proyecto.vista`

Clase	Descripción
<code>FrmLogin</code>	Ventana de inicio de sesión con fondo degradado y logo institucional. Valida las credenciales contra <code>UsuarioDao</code> y, de ser correctas, abre <code>VentanaPrincipal</code> .
<code>VentanaPrincipal</code>	Ventana principal de tipo MDI (Multiple Document Interface) implementada con <code>JDesktopPane</code> ; cada módulo se abre como una <code>JInternalFrame</code> independiente, gestionadas mediante un mapa (<code>ventanasAbiertas</code>) para evitar duplicados.
<code>DashboardInterna</code>	Panel interno de inicio que se muestra automáticamente al ingresar, a modo de resumen general del sistema.
<code>PaisInterna</code> , <code>DepartamentoInterna</code> , <code>CargoInterna</code> , <code>ProyectoInterna</code>	Ventanas internas CRUD para sus respectivas entidades, con tabla (<code>JTable</code>), formulario y botones de acción.
<code>EmpleadoInterna</code>	Módulo CRUD de empleados; incluye combos para seleccionar país, departamento y cargo, y opciones de exportación.
<code>AsignacionInterna</code>	Módulo CRUD de asignaciones; permite vincular empleados con proyectos y registrar las horas asignadas.
<code>EstiloUI</code>	Clase de utilidades que centraliza colores, fuentes y el Look and Feel de toda la aplicación, garantizando consistencia visual.
<code>UtilidadesUI</code>	Métodos auxiliares reutilizables para la construcción de componentes de interfaz.
<code>ExportadorCSV</code>	Exporta el contenido de una <code>JTable</code> a un archivo CSV codificado en UTF-8 con BOM, separado por punto y coma.
<code>ExportadorPDF</code>	Exporta el contenido de una <code>JTable</code> a un archivo PDF con encabezado, título y estilo de tabla, utilizando la librería <code>iText 5</code> .

9. Interfaz gráfica y navegación

La interfaz gráfica se construyó íntegramente con Swing, sin frameworks adicionales, priorizando un estilo visual oscuro y moderno definido de forma centralizada en EstiloUI.

9.1 Flujo de navegación

1. Main.java inicia la aplicación y muestra FrmLogin.
2. El usuario ingresa sus credenciales (usuario / contraseña).
3. UsuarioDao.validarAcceso() consulta la tabla usuarios.
4. Si el acceso es válido, se abre VentanaPrincipal y se cierra FrmLogin.
5. VentanaPrincipal muestra DashboardInterna de forma automática y maximizada.
6. Desde el menú "Módulos", el usuario abre cualquiera de las ventanas internas: Países, Departamentos, Cargos, Empleados, Proyectos o Asignaciones, cada una gestionada como una JInternalFrame dentro del JDesktopPane.

9.2 Módulo de exportación

Cada ventana de listado (JTable) ofrece la opción de exportar su contenido visible a CSV mediante ExportadorCSV, o a PDF mediante ExportadorPDF. La exportación a PDF genera un documento en formato horizontal (A4 apaisado) con un encabezado de título, la fecha de generación y una tabla con las filas y columnas visibles, replicando el color corporativo (azul, RGB 59, 130, 246) definido en la interfaz.

10. Manual de instalación y despliegue

10.1 Requisitos previos

- Java Development Kit (JDK) 17 o superior instalado.
- Apache Maven 3.8 o superior instalado y disponible en el PATH.
- Acceso de escritura en la carpeta del proyecto (para la creación del archivo organizacion.db).

10.2 Compilación del proyecto

Desde la raíz del proyecto (donde se encuentra el archivo pom.xml), ejecutar:

```
mvn clean package
```

Este comando compila el código fuente, descarga las dependencias declaradas en pom.xml (sqlite-jdbc e itextpdf) y, gracias al plugin maven-shade-plugin, genera un archivo JAR ejecutable autocontenido en la carpeta target, con el nombre ProyectoFinalPOO.jar, que incluye todas las dependencias necesarias.

10.3 Ejecución de la aplicación

Una vez generado el JAR, la aplicación puede ejecutarse con:

```
java -jar target/ProyectoFinalPOO.jar
```

Alternativamente, durante el desarrollo, puede ejecutarse directamente la clase Main desde el IDE (IntelliJ IDEA, Eclipse o NetBeans) importando el proyecto como un proyecto Maven existente.

10.4 Primer inicio

En el primer arranque, ConexionSQLite crea automáticamente el archivo organizacion.db (si no existe) junto con todas las tablas del sistema, y registra el usuario administrador por defecto:

Campo	Valor por defecto
Usuario	admin
Contraseña	1234
Rol	Administrador

Se recomienda, en un entorno de uso real, cambiar estas credenciales o implementar un mecanismo de cifrado de contraseñas, dado que en la versión actual se almacenan en texto plano con fines académicos.

10.5 Ubicación de la base de datos

El archivo organizacion.db se crea en el directorio desde el cual se ejecuta la aplicación (directorio de trabajo del proceso Java), tal como se define en ConexionSQLite mediante la URL de conexión jdbc:sqlite:organizacion.db.

11. Manejo de errores y validaciones

El sistema distingue dos categorías de errores durante las operaciones CRUD:

- Errores de validación de negocio: representados por `ValidacionException`, lanzados por los controladores cuando los datos ingresados no cumplen las reglas descritas en la sección 8.5 (campos obligatorios, rangos numéricos, duplicados).
- Errores técnicos de persistencia: representados por `SQLException`, propagados desde la capa DAO ante fallos de conexión, restricciones de integridad referencial u otros errores del motor de base de datos.

En ambos casos, la capa de vista captura estas excepciones de forma independiente y presenta al usuario un mensaje descriptivo mediante `JOptionPane`, evitando que la aplicación se detenga de forma abrupta ante un dato inválido o un error de base de datos.

11.1 Ejemplo de manejo en la capa de vista

```
try {
    controlador.guardar(empleado);
    // refrescar tabla / mostrar mensaje de éxito
} catch (ValidacionException ve) {
    JOptionPane.showMessageDialog(this, ve.getMessage(),
        "Datos inválidos", JOptionPane.WARNING_MESSAGE);
} catch (SQLException se) {
    JOptionPane.showMessageDialog(this, "Error de base de datos: " + se.getMessage(),
        "Error", JOptionPane.ERROR_MESSAGE);
}
```

12. Conclusiones

El Sistema de Gestión Organizacional cumple con el objetivo académico de demostrar, en una aplicación funcional y completa, los principios fundamentales de la Programación Orientada a Objetos: encapsulamiento en todas las clases del modelo, herencia entre Persona y Empleado, polimorfismo mediante la sobrescritura de `getDescripcion()`, y composición en las clases Empresa y Asignacion.

Adicionalmente, la arquitectura en capas (modelo, dao, controlador, vista) y el uso del patrón Singleton para la gestión de la conexión a base de datos reflejan buenas prácticas de diseño de software que facilitan el mantenimiento, la extensión y la comprensión del sistema por parte de terceros.

Como posibles mejoras futuras se identifican: el cifrado de contraseñas de usuario, la incorporación de un framework de pruebas unitarias (JUnit) para la capa de controladores y DAO, y la externalización de la ruta de la base de datos mediante un archivo de configuración.